

SILICA FLUX ASIC

A Parameterised Hardware Architecture for High-Performance Proof-of-Work Mining Silicon

Technical White Paper

Abstract

Silica Flux ASIC is a proposed hardware architecture and development methodology for specialised proof-of-work mining silicon. The system treats cryptographic mining not as a software workload to be translated into hardware, but as a physical computational architecture that can be systematically decomposed, verified, synthesised and optimised.

The architecture combines three principal computational layers. **Python** provides an independent mathematical reference implementation and golden verification model. **SystemVerilog** implements the synthesizable digital hardware, including SHA-256 datapaths, pipeline structures, nonce engines, work distribution and mining-core replication. **R** provides the analytical layer for evaluating area, timing, power, throughput and design-space trade-offs.

The proposed architecture progresses through ten stages: mathematical verification, synthesizable cryptographic RTL, double SHA-256, nonce generation, target validation, single-core mining, pipeline and unrolling exploration, multi-core replication, ASIC synthesis and final ASIC-oriented optimisation.

Rather than defining a single fixed miner design, Silica Flux defines a **parameterised architecture space**. Pipeline depth, SHA-256 unrolling, core count, clock frequency and power-management mechanisms become design variables. This allows alternative hardware organisations to be evaluated against measurable objectives such as hashes per second, hashes per joule and hashes per unit silicon area.

The resulting framework is intended as both a research architecture and a practical foundation for ASIC-oriented hardware development.

1. Introduction

Proof-of-work mining is an unusual computational workload.

The underlying cryptographic algorithm is deterministic and highly structured, while the search space is extraordinarily large. The workload is therefore particularly amenable to specialised hardware: once the mathematical transformation is understood, much of the computational process can be reduced to fixed-width arithmetic, logical operations, registers and highly regular data paths.

This makes proof-of-work mining an interesting case study in **application-specific integrated circuit architecture**.

A general-purpose processor attempts to execute a cryptographic algorithm using a relatively flexible computational substrate. An ASIC instead removes much of that flexibility and constructs hardware specifically around the operations required by the algorithm.

The resulting engineering problem is therefore not simply:

How can SHA-256 be implemented in hardware?

It is:

How should SHA-256 computation be physically organised to maximise useful cryptographic throughput within constraints imposed by area, timing, power, thermal density and verification?

Silica Flux ASIC is proposed as a framework for investigating this question.

2. Design Philosophy

The fundamental principle of Silica Flux is:

Design the computation as silicon rather than treating silicon as a destination for software.

This principle affects the entire architecture.

A software-oriented implementation might be represented as:

```
for nonce in range:
    hash(block_header + nonce)
    if hash < target:
        return nonce
```

An ASIC implementation instead consists of physical structures:

```
JOB
|
v
NONCE GENERATOR
|
v
MESSAGE PREPARATION
|
v
SHA-256 DATAPATH
|
v
SHA-256 DATAPATH
|
v
TARGET COMPARATOR
|
v
RESULT
```

Thousands of these operations may eventually exist concurrently across a physical array.

Consequently, the design must consider:

- combinational depth;
- sequential state;
- register placement;
- pipeline latency;
- clock frequency;
- fanout;
- switching activity;
- core replication;
- work distribution;
- routing;
- thermal behaviour;
- area;
- power.

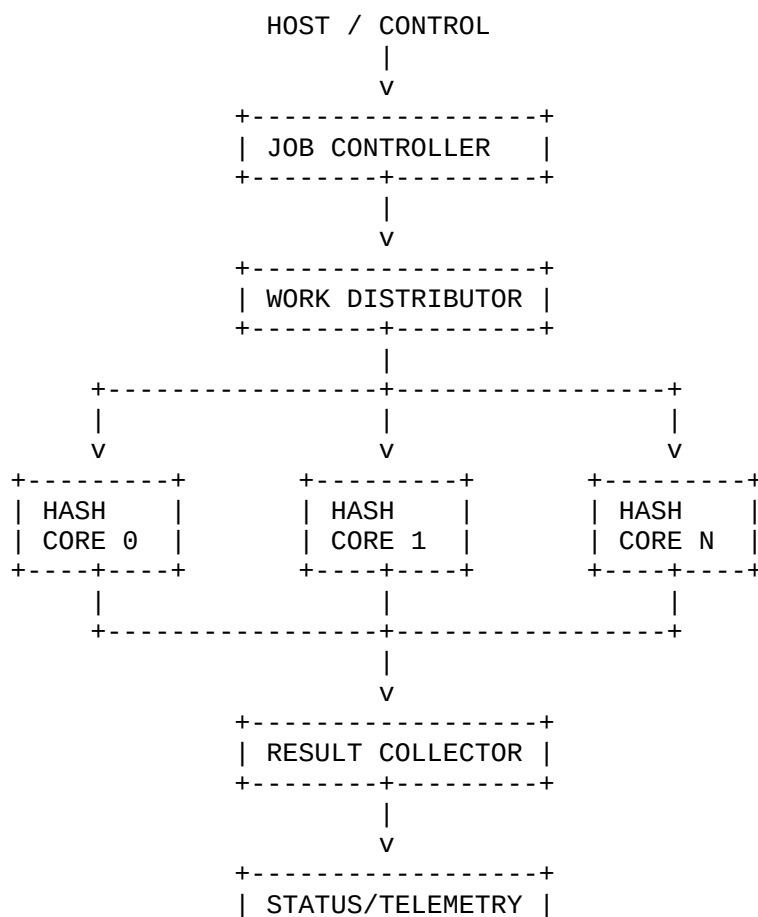
Silica Flux therefore separates **algorithmic correctness** from **architectural optimisation**.

The governing development sequence is:

Correctness → Verification → Synthesis → Measurement → Optimisation

3. System Architecture

The proposed top-level architecture is:



+-----+

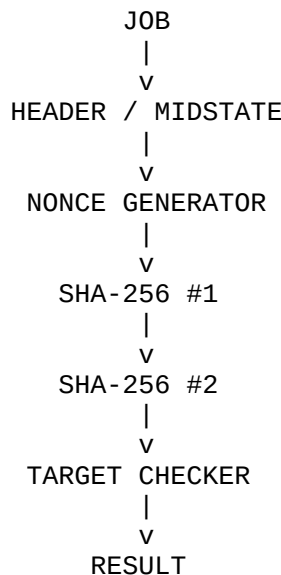
The system is divided into five major functional domains:

1. **Control**
2. **Work distribution**
3. **Cryptographic computation**
4. **Result validation**
5. **Telemetry**

This separation allows individual components to be independently tested and subsequently replicated or replaced.

4. Hash-Core Architecture

The fundamental computational unit is the hash core.



The hash core should be independent of the host protocol.

Its responsibilities are restricted to the computational mining process:

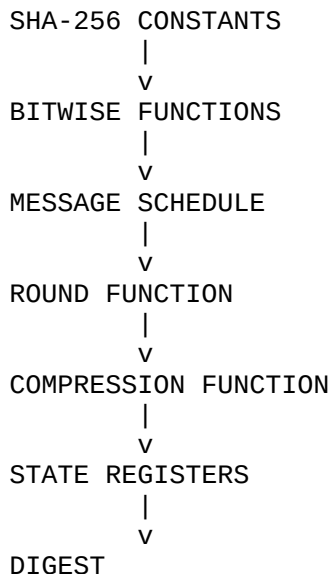
- receiving work;
- generating candidate nonces;
- constructing the appropriate input;
- calculating the cryptographic transformation;
- comparing the result with the target;
- reporting successful results;
- reporting exhaustion.

This creates a clean architectural boundary between **mining computation** and **external system control**.

5. SHA-256 Hardware Engine

SHA-256 is decomposed into explicit hardware structures.

The proposed RTL hierarchy includes:



The hardware must explicitly represent the cryptographic transformations rather than relying upon behavioural software abstractions.

Important implementation concerns include:

- 32-bit word arithmetic;
- modular addition;
- logical rotations;
- logical shifts;
- message scheduling;
- round constants;
- state accumulation;
- padding;
- byte ordering;
- word ordering.

Every stage must have an independently testable interface.

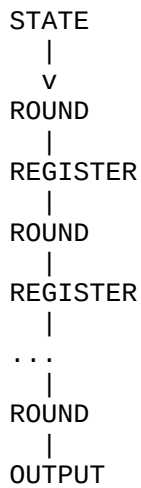
6. SHA-256 Architectural Variants

One of the central features of Silica Flux is that SHA-256 is treated as a **parameterised hardware architecture**.

Three principal implementations are proposed.

6.1 Iterative Architecture

The simplest architecture reuses one round-processing unit.



The same hardware performs multiple rounds over multiple clock cycles.

Advantages

- low silicon area;
- comparatively simple control;
- lower hardware duplication;
- potentially lower instantaneous switching activity.

Disadvantages

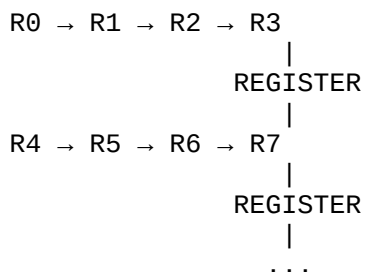
- high cycles per hash;
- relatively low throughput;
- increased control overhead per result.

The iterative architecture provides the fundamental correctness baseline.

7. Partially Unrolled Architecture

A partially unrolled design instantiates multiple SHA-256 rounds physically.

For example:



The number of rounds per stage becomes a design parameter.

Potential configurations include:

- 2;
- 4;
- 8;

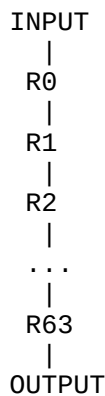
- 16;
- 32 rounds per physical section.

This produces an architectural continuum between the compact iterative design and the fully pipelined architecture.

The key research question becomes whether increased physical parallelism produces sufficient frequency and throughput improvements to justify the additional silicon and power.

8. Fully Pipelined Architecture

The fully pipelined architecture physically represents the SHA-256 round sequence as a pipeline.



Once filled, the pipeline can potentially accept a new work item every cycle.

This changes the primary optimisation problem.

Instead of minimising cycles per hash, the design attempts to maximise:

Hashes per cycle

while maintaining a sufficiently high clock frequency.

The resulting architecture introduces additional costs:

- pipeline registers;
- clock-tree load;
- switching activity;
- area;
- routing complexity;
- physical timing constraints.

The fully pipelined implementation is therefore not assumed to be optimal.

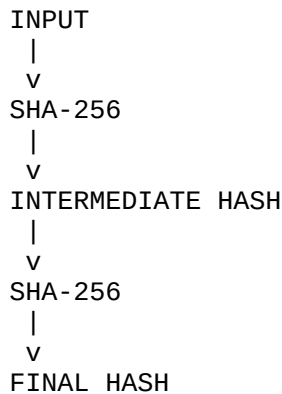
It is one point within the Silica Flux design space.

9. Double SHA-256

The mining datapath requires a double application of SHA-256:

$H = \text{SHA256}(\text{SHA256}(M))$

The architecture therefore includes a dedicated double-hash layer.



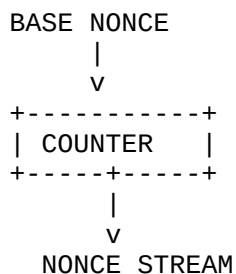
The cryptographic layer should remain generic enough to support independent verification of both transformations.

Mining-specific assumptions should be introduced only at the hash-core level.

This separation permits the SHA-256 hardware to be reused for other applications.

10. Nonce Generation

The nonce engine is a distinct hardware subsystem.



The nonce engine must provide:

- configurable starting point;
- configurable limit;
- deterministic increment;
- enable;
- reset;
- exhaustion detection.

Its behaviour should be formally describable.

For a sequential nonce search:

$n_{i+1} = n_i + 1$

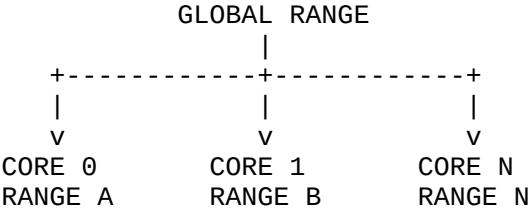
subject to defined enable, reset and exhaustion conditions.

This allows the nonce engine to be tested independently from the cryptographic datapath.

11. Work Distribution

As the architecture scales beyond a single hash core, nonce allocation becomes a fundamental hardware problem.

The work distributor receives a global search region and divides it between cores.



The ranges must be deterministic and non-overlapping.

For distinct cores i and j :

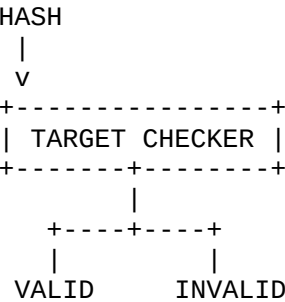
$$R_i \cap R_j = \emptyset$$

under normal allocation conditions.

This property is suitable for formal verification.

12. Target Comparison

The target checker converts the cryptographic output into a proof-of-work decision.



The fundamental relationship is:

$$\text{valid} \iff \text{hash} < \text{target}$$

subject to the precise numerical and byte-order conventions of the target representation.

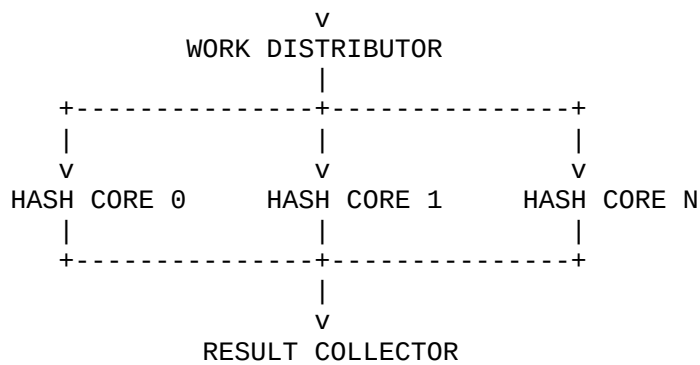
The architecture must not substitute a simplistic leading-zero test for numerical target comparison.

The target checker should be independently verified against Python reference calculations.

13. Multi-Core Architecture

Once a single hash core is verified, the architecture becomes scalable through replication.





The number of hash cores is represented by a parameter:

Ncores

This permits experiments involving:

1,2,4,8,16,32,...

cores without fundamentally changing the architecture.

However, replication introduces new constraints.

Hashrate may scale approximately with core count under ideal conditions, but physical implementation may introduce:

- routing congestion;
- power growth;
- clock distribution costs;
- work-distribution overhead;
- thermal constraints;
- diminishing returns.

Consequently:

$H_{\text{?}} = N_{\text{cores}} \times H_{\text{core}}$

in every physical implementation.

The purpose of synthesis and analysis is to determine the deviation from the ideal model.

14. Hardware Telemetry

Silica Flux includes telemetry as part of the architecture.

The hardware should measure:

- total cycles;
- hashes attempted;
- successful results;
- failed target comparisons;
- exhausted ranges;
- idle cycles;
- core utilisation;

- error conditions.

This permits actual hardware behaviour to be compared with theoretical expectations.

For example:

Utilisation = $\frac{\text{Total Cycles}}{\text{Active Cycles}}$

and:

Observed Hashrate = $\frac{\text{Elapsed Time}}{\text{Hashes}}$

These measurements become particularly important once multiple cores and deep pipelines are introduced.

15. Python Reference Layer

Python acts as the independent mathematical authority.

The Python system provides:

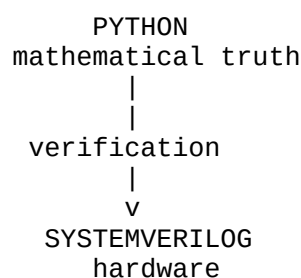
SHA-256
Double SHA-256
Block-header generation
Nonce generation
Target comparison
Mining reference model
Test-vector generation

The Python implementation is intentionally independent of the RTL.

Its purpose is not performance.

Its purpose is correctness.

The relationship is:



This prevents an RTL implementation from becoming self-validating.

16. SystemVerilog Hardware Layer

SystemVerilog is the principal hardware implementation language.

The RTL hierarchy should eventually contain:

```
rtl/
  flux_pkg.sv
```

```

crypto/
  sha256_constants.sv
  sha256_functions.sv
  sha256_round.sv
  sha256_message_schedule.sv
  sha256_compression.sv
  sha256_core.sv
  sha256_double_hash.sv
mining/
  nonce_counter.sv
  nonce_allocator.sv
  target_checker.sv
  hash_core.sv
  miner_core.sv
  work_distributor.sv
  result_collector.sv
control/
  job_controller.sv
  control_registers.sv
  status_registers.sv
telemetry/
  hash_counter.sv
  cycle_counter.sv
  error_counter.sv
  performance_counter.sv
top/
  miner_top.sv

```

This hierarchy creates a direct correspondence between architectural concepts and hardware modules.

17. Verification Architecture

Verification proceeds at multiple levels.

Unit verification

Individual cryptographic and control modules are tested independently.

Integration verification

Hash cores and mining cores are tested as complete datapaths.

System verification

The multi-core mining array is tested with complete jobs.

Formal verification

Invariants are checked independently of simulation traces.

The verification hierarchy is therefore:

```

Known-answer tests
|

```

Python reference

|

v

RTL simulation

|

v

Randomised testing

|

v

Assertions

|

v

Formal verification

Every discovered defect should become a permanent regression test.

18. Formal Properties

Important properties include:

Reset

Reset must produce deterministic state.

Nonce progression

The nonce must advance correctly when enabled.

Nonce uniqueness

A single core must not accidentally repeat a nonce.

Work separation

Distinct cores must not receive overlapping ranges.

Pipeline validity

Invalid pipeline data must not generate valid results.

Target comparison

$\text{valid} \iff \text{hash} < \text{target}$

Control

Start, stop, busy and result states must remain deterministic.

Formal verification therefore becomes an architectural tool rather than simply a final-stage verification mechanism.

19. Power Efficiency

ASIC mining cannot be evaluated purely through hash rate.

A more useful metric is:

$$\eta_H = \frac{H}{P}$$

where:

- H is hash rate;
- P is power.

The result is measured as hashes per joule.

Silica Flux therefore investigates mechanisms including:

- clock enables;
- idle-core suppression;
- workload gating;
- pipeline activity reduction;
- result-triggered shutdown.

The design objective becomes:

maximise useful cryptographic computation per unit of energy.

20. Area Efficiency

A second important metric is computational density.

Define:

$$\eta_A = \frac{H}{A}$$

where:

- H = hash rate;
- A = silicon area.

This gives a measure of hashes per unit area.

The two efficiency metrics can conflict.

A highly parallel architecture may achieve excellent hash rate per second while consuming considerable silicon.

A compact iterative architecture may use much less silicon while producing significantly less throughput.

The optimal architecture therefore depends on the design objective.

21. Design-Space Exploration

The central research capability of Silica Flux is the ability to vary architectural parameters.

A simplified parameter vector is:

$$D=(N,U,F)$$

where:

- N = number of hash cores;
- U = SHA-256 unrolling/pipeline configuration;
- F = operating frequency.

Additional variables may include:

- clock-gating strategy;
- register placement;
- work-distribution architecture;
- target-check architecture.

The resulting performance can be represented as:

$$H=f(N,U,F)$$

while power can be represented as:

$$P=g(N,U,F)$$

and area as:

$$A=h(N,U,F)$$

The purpose of the experimental framework is to estimate or measure these functions rather than assume them.

22. R Analytical Layer

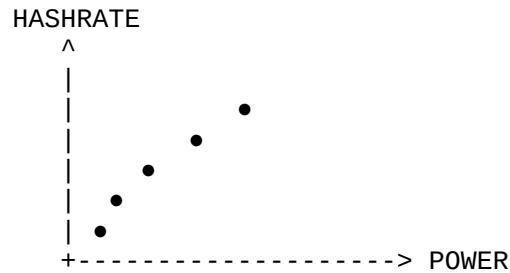
R provides the final comparative analysis.

The analytical system should accept simulation and synthesis results and calculate:

- hash rate;
- area;
- power;
- timing;
- hashes per joule;
- hashes per square millimetre;
- cycles per hash;
- hashes per cycle;
- utilisation.

Architectures can then be plotted against one another.

For example:



The resulting Pareto frontier identifies architectures for which no other design simultaneously provides superior performance and lower resource consumption.

23. ASIC Synthesis

The transition from RTL to ASIC requires explicit synthesis constraints.

The project therefore introduces:

```
constraints/miner.sdc  
synthesis/synthesis.tcl
```

The synthesis stage should measure, where supported:

- cell area;
- sequential area;
- combinational area;
- critical path;
- maximum frequency;
- slack;
- fanout;
- buffering;
- estimated power.

The methodology makes an explicit distinction between:

modelled results

simulated results

synthesised results

measured silicon results

This distinction is critical.

A simulated 1 GHz design is not a 1 GHz ASIC.

Likewise, an RTL area estimate is not a physical silicon-area measurement.

24. ASIC-Oriented Optimisation

Once synthesis results are available, optimisation begins.

Areas of investigation include:

- critical datapaths;
- pipeline placement;
- high-fanout signals;
- register duplication;
- combinational duplication;
- clock enables;
- reset distribution;
- target-comparison timing;
- nonce distribution;
- work-distribution overhead;
- result arbitration;
- replicated-core placement.

At later stages, physical design introduces additional concerns:

- clock-tree synthesis;
- routing congestion;
- floorplanning;
- thermal density;
- voltage/frequency relationships;
- physical locality.

The architecture should therefore evolve based upon physical evidence.

25. Ten-Phase Development Methodology

Silica Flux defines a ten-phase progression.

Phase 1 — Mathematical Foundation

Develop and verify the independent SHA-256 Python model.

Phase 2 — Synthesizable RTL

Implement the iterative SHA-256 hardware.

Phase 3 — Double SHA-256

Construct the mining-compatible cryptographic datapath.

Phase 4 — Nonce Engine

Develop deterministic nonce generation and allocation.

Phase 5 — Target Checker

Implement numerical proof-of-work validation.

Phase 6 — Single Mining Core

Integrate the complete mining datapath.

Phase 7 — Pipeline and Unrolling

Explore alternative SHA-256 hardware architectures.

Phase 8 — Multi-Core Array

Replicate cores and distribute work.

Phase 9 — ASIC Synthesis

Measure timing, area and power.

Phase 10 — ASIC Optimisation

Optimise the architecture using physical and analytical evidence.

This creates a controlled progression from abstract mathematics to specialised silicon.

26. Codebase Foundation

The final repository is structured as:

```
silica-flux-asic/  
├── rtl/  
│   ├── flux_pkg.sv  
│   ├── crypto/  
│   ├── mining/  
│   ├── control/  
│   ├── telemetry/  
│   └── top/  
├── tb/  
│   ├── crypto/  
│   ├── mining/  
│   └── integration/  
├── formal/  
├── python/  
│   ├── reference/  
│   ├── vectors/  
│   ├── simulation/  
│   └── optimisation/  
├── r/  
├── constraints/  
└── synthesis/
```

```
|
|— scripts/
|— docs/
```

The codebase is intentionally modular.

The cryptographic engine can be replaced without rewriting the work distributor.

The nonce engine can be replaced without rewriting the SHA-256 engine.

The number of cores can change without fundamentally changing the core architecture.

This makes the system suitable for systematic experimentation.

27. Architecture Versions

The development can be represented through architectural generations:

Version	Architecture
SF-A0	Python reference
SF-A1	Iterative SHA-256
SF-A2	Double SHA-256
SF-A3	Nonce engine
SF-A4	Target checker
SF-A5	Single mining core
SF-A6	Pipelined/unrolled engines
SF-A7	Multi-core array
SF-A8	Synthesis architecture
SF-A9	Power-aware architecture
SF-A10	ASIC-oriented architecture

Earlier architectures should be retained.

They provide experimental baselines against which subsequent designs can be compared.

28. Performance Metrics

Every architecture should be characterised using a consistent set of measurements.

Parameter	Description
Core count	Number of parallel mining engines
Pipeline depth	Number of pipeline stages
Rounds/stage	SHA-256 unrolling
Frequency	Operating clock
Cycles/hash	Cryptographic latency
Hashes/cycle	Throughput
Hashrate	Hashes per second

Parameter	Description
Area	Silicon area
Power	Electrical consumption
Hashes/watt	Energy efficiency
Hashes/mm ²	Computational density
Utilisation	Active computation percentage

These measurements provide a common language for comparing architectures.

29. Research Questions

The Silica Flux architecture creates several research questions.

Question 1

What degree of SHA-256 unrolling produces the best throughput/area trade-off?

Question 2

At what point does core replication produce diminishing returns?

Question 3

How does pipeline depth influence frequency, power and routing complexity?

Question 4

How much energy efficiency can be gained through hardware-level activity management?

Question 5

What work-distribution architecture minimises control overhead in a highly replicated design?

Question 6

What architecture lies on the Pareto frontier between:

Hashrate, Area, Power

?

These questions transform the project from a simple ASIC implementation into a hardware architecture research platform.

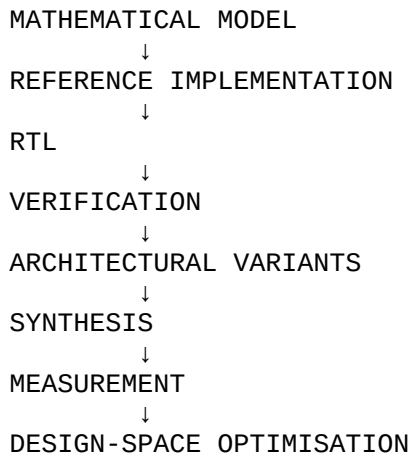
30. Broader Applicability

Although the initial architecture is designed around proof-of-work mining, the underlying methodology is broader.

The same architecture could be applied to specialised accelerators for:

- cryptographic workloads;
- zero-knowledge computation;
- hashing;
- search problems;
- scientific computing;
- signal processing;
- numerical kernels;
- other deterministic high-throughput workloads.

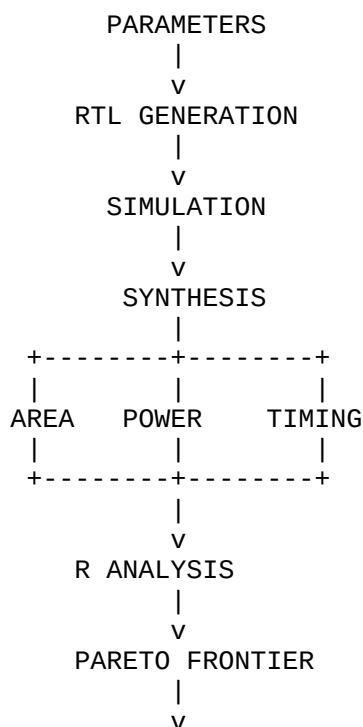
The general methodology remains:



Silica Flux is therefore better understood as a **methodology for computational silicon** than merely as a mining-chip design.

31. Future Development

Future versions could introduce an automated hardware exploration loop.



NEXT ARCHITECTURE

This could ultimately transform the project into a semi-automated hardware architecture laboratory.

A parameter set could be generated, synthesised, measured and evaluated automatically, allowing thousands of candidate architectures to be explored computationally.

The result would be a hardware equivalent of automated model selection.

32. Conclusion

Silica Flux ASIC proposes a systematic architecture for developing specialised proof-of-work mining silicon.

The project is founded on a simple principle:

The ASIC should be designed as a computational architecture, not merely as an implementation target for an algorithm.

Python establishes independent mathematical correctness.

SystemVerilog establishes physical digital computation.

Formal verification establishes behavioural invariants.

Synthesis establishes measurable hardware constraints.

R establishes the analytical framework through which competing architectures can be compared.

The resulting system transforms ASIC mining development from a single implementation exercise into a structured design-space exploration problem.

The central optimisation problem can ultimately be expressed as:

Optimise (PH,AH,H)

subject to:

Timing, Area, Power, Thermal, Routing, Verification

constraints.

The objective is therefore not simply to build the fastest miner.

It is to discover the **best physical organisation of cryptographic computation for a defined silicon, timing and energy envelope.**

That is the central proposition of **Silica Flux ASIC.**